



CSS - Cascading Style sheetS



- Standard W3C
 - linguaggio indipendente con la sua sintassi
- Perché esiste il CSS
 - Definisce la presentazione del documento HTML (o in generale XML)
 - Stesso HTML, presentazioni diverse;
 - CSS separa contenuto e resa visiva.
 - <https://csszengarden.com/pages/alldesigns/>
 - Uniforma la visualizzazione tra i browser



Come funziona CSS

- HTML struttura il contenuto
- CSS descrive la presentazione
- Il browser combina HTML + CSS per visualizzare la pagina

```
<h1>Benvenuti</h1>  
<p>Questo è un paragrafo.</p>
```

```
h1 { color: darkred; }  
p { font-size: 20px; }
```

Dove si scrive CSS

- External style sheet

- collegato mediante il tag `<link>`
- file txt con estensione .css dove scriviamo le regole

```
<head>
<link rel="stylesheet" type="text/css" href="mystyle.css">
</head>
```

- Internal style element

- regole nell'head tra i tag
`<style>...</style>`

mystyle.css > ...

```
1  body {
2      background-color: lightblue;
3  }
4
5  h1 {
6      color: navy;
7      margin-left: 20px;
8  }
```

```
<head>
<style>
body {
    background-color: linen;
}
h1 {
    color: maroon;
    margin-left: 40px;
}
</style>
</head>
```



Elemento <link>

- Allega un file alla pagina corrente
 - va posizionato nella sezione head del file html

- **CSS**

```
<head>  
  <link rel="stylesheet" type="text/css" href="theme.css">  
</head>
```

- **Favicon**

```
<link rel="icon" href="demo_icon.gif" type="image/gif">
```



Dove si scrive CSS: Inline

```
<!DOCTYPE html>
<html>
<head>
</head>
<body>
  <h1 style="color:red;">This is a Heading</h1>
  <p style="font-size: 1.1em; font-family: sans-serif;">This is a
paragraph.</p>
</body>
</html>
```

- Definisco lo stile usando l'attributo style
 - Viene applicata solo all'elemento
 - dichiarazioni multiple separate con ;
- Si usa in casi molto particolari
 - per fare override mirati di regole esterne



Com'è fatta una regola CSS

```
h1 {  
  color: green;  
  font-size: 32px;  
}
```

- **h1** → selettore
- **color: green;** → dichiarazione
- **font-size: 32px;** → dichiarazione
- **color / font-size** → proprietà
- **green / 32px** → valore

declaration block

```
selector {  
  property1: value1;  
  property2: value2;  
  property3: value3;  
}
```

Una regola dice: quali elementi selezionare e quali proprietà applicare!!

Il browser applica le regole agli elementi



```
<h1>Titolo</h1>  
<p>Primo paragrafo</p>  
<p>Secondo paragrafo</p>  
<div>Un div</div>
```

```
p {  
  color: blue;  
}
```

- i due paragrafi cambiano
- h1 e div no
- il selettore decide il "bersaglio"
- le regole cambiano la visualizzazione



Primo selettore: il nome dell'elemento

- Seleziono tutti gli elementi di quel tipo

Esempio: seleziono tutti i **p**

```
p {  
    color: red;  
    text-align: center;  
}
```

```
body {  
    background-color: lightblue;  
}
```

```
h1 {  
    color: navy;  
    margin-left: 20px;  
}
```

```
p{  
    font-size: large;  
}
```

```
section{  
    margin-top: 10px;  
}
```



Selettore di classe

- Seleziona tutti gli elementi che appartengono a una classe
 - In HTML la classe si assegna con l'attributo class
 - In CSS si usa il nome preceduto da un punto

Esempio

```
<h1 class="center">Heading</h1>  
<p class="center">A paragraph</p>
```

```
.center {  
  text-align: center;  
  color: red;  
}
```

```
p.center {  
  text-align: center;  
  color: green;  
}
```

Elementi diversi possono condividere la stessa classe

https://www.w3schools.com/css/tryit.asp?filename=trycss_syntax_class

https://www.w3schools.com/css/tryit.asp?filename=trycss_syntax_element_class



Selettore id

- Seleziona un elemento specifico
 - In HTML si usa l'attributo id
 - In CSS si scrive con il simbolo #
 - L'id dovrebbe essere unico nella pagina

- Esempio:

```
<p id="intro">Benvenuti</p>
```

```
#intro {  
    color: darkgreen;  
}
```

https://www.w3schools.com/css/tryit.asp?filename=trycss_syntax_id



Elemento, classe, id

- Elemento → tutti i tag di un tipo
- Classe → tutti gli elementi con una certa classe
- ID → un solo elemento specifico

```
<h1 class="warning">Titolo</h1>  
<p id="intro">Introduzione</p>  
<p class="warning">Messaggio importante</p>  
<p>Testo normale</p>
```

```
p { color: #444; }  
.warning { color: firebrick; }  
#intro { background: #f0f8e8; }
```

Scegliamo il selettore in base a quanto è generale o specifico il bersaglio

Esempio



```
<article>
  <p>Ciao a tutti</p>
  <p>Abbiamo iniziato il CSS</p>
  <p>come fai a cambiare lo stile a me?</p>
  <p>e me no?</p>
  <p>ma me si!!!</p>
  <p>Io voglio uno stile tutto mio</p>
  <section>
    <p>non mi scordate di me
      dentro la section</p>
  </section>
</article>
```

- Quale selettore CSS usereste per modellare il contenuto del solo 6° <p> tag?
- Quale selettore CSS usereste per modellare il contenuto del 3° e 5° <p> tag insieme?

Esempio



```
<article>
  <p>Ciao a tutti</p>
  <p>Abbiamo iniziato il CSS</p>
  <p class="stile-cl">come fai a cambiare lo stile a me?</p>
  <p>e me no?</p>
  <p class="stile-cl">ma me si!!!</p>
  <p id="stile-id">Io voglio uno stile tutto mio</p>
  <section>
    <p>non mi scordate di me
      dentro la section</p>
  </section>
</article>
```

```
.stile-cl{
  color: ■red;
}

#stile-id{
  color: ■green;
}
```



Raggruppare selettori

- Si possono anteporre più selettori ad un blocco di dichiarazioni separati dalla virgola

```
<h1>Titolo principale</h1>  
<h2>Sottotitolo</h2>  
<h3>Sezione</h3>  
<p>Paragrafo</p>
```

```
h1, h2, h3 {  
    font-family: Georgia, serif;  
    color: darkslateblue;  
}
```

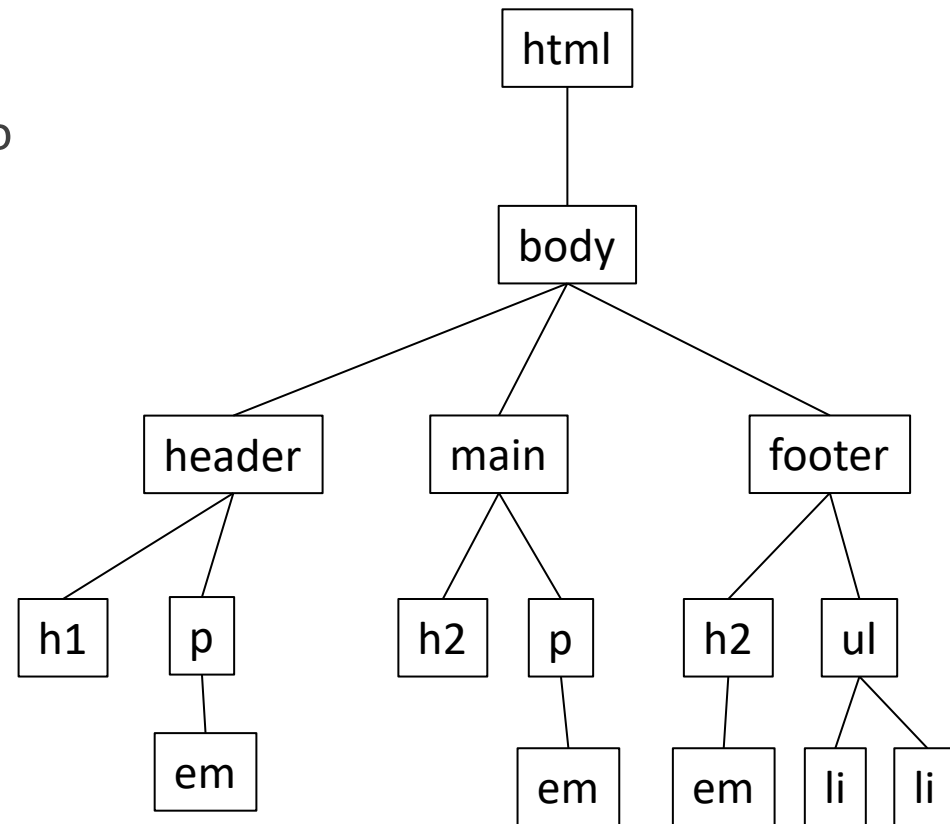


Selettori E DOM



HTML come albero di elementi

- **Descendant** - discendenti
 - Elementi contenuti in un elemento
- **Child** - figli
 - Discendenti diretti
- **Ancestor** - antenati
 - Gli elementi sopra nell'albero
- **Parent** – genitore
 - Elementi direttamente sopra
- **Sibling** – fratelli
 - Elementi con lo stesso parent



Per CSS non conta solo il tipo di elemento, ma anche dove si trova nell'albero HTML



Selettori composti

1. Selettori per descendant (spazio)

$A B \rightarrow$ seleziona i discendenti B di A

2. Selettori per child (>)

$A > B \rightarrow$ seleziona i figli diretti B di A

3. Selettori per Adjacent sibling (+)

$A + B \rightarrow$ seleziona il fratello immediatamente successivo

4. Selettori per General sibling (~)

$A \sim B \rightarrow$ seleziona tutti i fratelli successivi

Esempio



```
<section>
  <h2>Titolo</h2>
  <p>Primo paragrafo</p>
  <p>Secondo paragrafo</p>
  <div>
    <p>Paragrafo interno al div</p>
  </div>
</section>
```

```
section p { color: blue; }
```

colpisce tutti i p dentro section, anche quello dentro div:

```
section > p { color: green; }
```

colpisce solo i due paragrafi figli diretti:

```
h2 + p { font-weight: bold; }
```

colpisce solo il primo p subito dopo h2:

Esercizio 1 -



`p em {color: grey;}`

`header p {font-size: 20px;}`

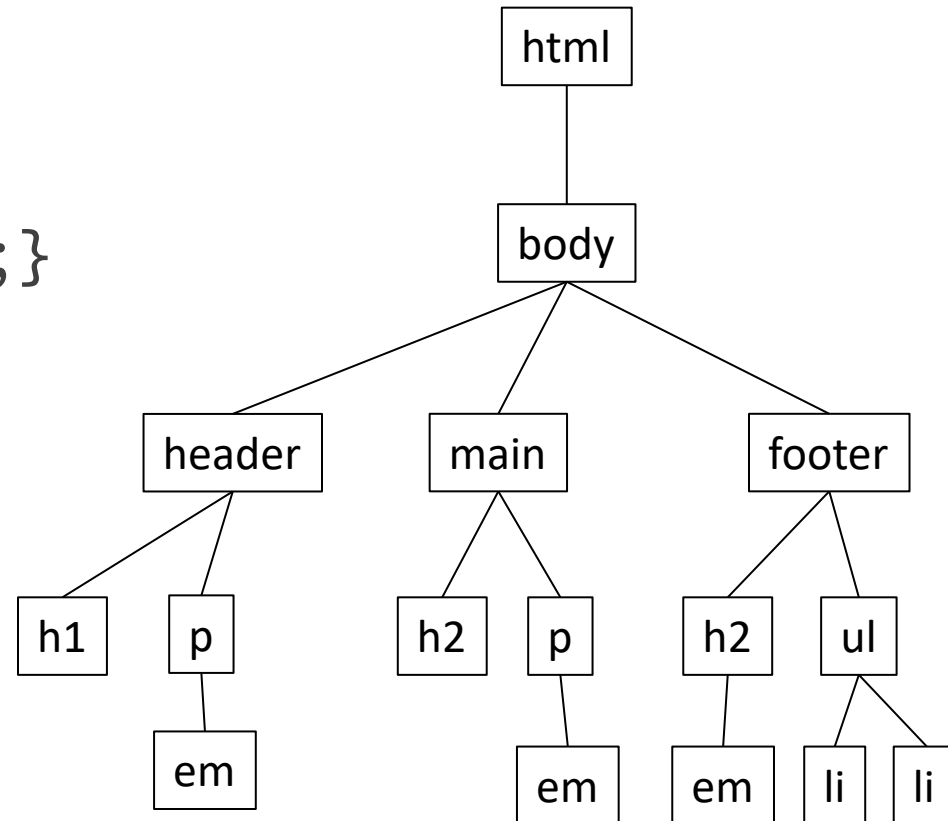
`main p {font-size: 16px;}`

`footer h2 {...}`

`main h2{...}`

`header em {...}`

`main em {...}`



Esercizio 2



p.ingredienti {...}
div.ricetta {...}

.ricetta p {...}
.ricetta h2 {...}

.ricetta .ingredienti {...}
div.ricetta p.ingredienti {...}

```
<div class="ricetta">  
  <h2>...</h2>  
  <p>...</p>  
  <p class="ingredienti"> ...</p>  
</div>  
  
<div class="ricetta">  
  <h2>...</h2>  
  <p>...</p>  
  <p class="ingredienti"> ...</p>  
</div>
```



Selettore: pseudo-classe

- Una pseudo-classe viene utilizzata per definire uno stato speciale di un elemento.
 - mouse over
 - visited e unvisited link
 - focus
 - posizione
 - ...

Esempio: stili ancora

https://www.w3schools.com/css/tryit.asp?filename=trycss_link

Elenco

https://www.w3schools.com/css/css_pseudo_classes.asp

```
selector:pseudo-class {  
    property:value;  
}
```

```
/* visited link */  
a:visited {  
    color: #00FF00;  
}
```

```
/* mouse over link */  
a:hover {  
    color: #FF00FF;  
}
```



Esempi: stato dell'utente

```
<a href="#">Apri dettaglio</a>  
<input type="text" placeholder="Scrivi qui">
```

```
a:hover {  
  color: crimson;  
}
```

```
input:focus {  
  border: 2px solid steelblue;  
}
```



Esempi: posizione dell'elemento

```
<ul>
  <li>Primo</li>
  <li>Secondo</li>
  <li>Terzo</li>
</ul>
```

```
li:first-child {
  font-weight: bold;
}
```

```
li:nth-child(2) {
  color: darkgreen;
}
```




Altre pseudo classi

- :root
- :only-child
- :empty
- :first-of-type
- :first-child
- :last-of-type
- :last-child
- :only-of-type
- :nth-child()
- :nth-last-child()
- :nth-of-type()
- :nth-last-of-type()

https://www.w3schools.com/css/css_pseudo_classes.asp



Pseudo elementi

- Permette di stilizzare una parte di un elemento
- Oppure di inserire contenuto prima o dopo

- Selettori

- ::after
- ::before
- ::first-letter
- ::first-line

```
selector::pseudo-element {  
  property:value;  
}
```

```
p::first-letter {  
  color: #ff0000;  
  font-size: xx-large;  
}
```

https://www.w3schools.com/css/css_pseudo_elements.asp

Esempi



```
<p>Questo paragrafo inizia con una lettera decorata.</p>
```

```
p::first-letter {  
  font-size: 2em;  
  color: darkred;  
  font-weight: bold;  
}
```

```
<p class="note">Messaggio importante</p>
```

```
.note::before {  
  content: "Attenzione: ";  
  color: crimson;  
  font-weight: bold;  
}
```



Selettori per attributi

- Non devo aggiungere per forza una classe: posso sfruttare attributi che l'elemento ha già

```
[attribute] {  
  property: value;  
}
```

```
[attribute=value] {  
  property: value;  
}
```

```
img[alt] {  
  background-color: grey;  
}
```

```
a[target="_blank"] {  
  color: red;  
}
```

- Come seleziono gli elementi:
 - parola, inizio, fine, etc.

https://www.w3schools.com/css/css_attribute_selectors.asp



Selector	Example	Example description
<u>[attribute]</u>	[target]	Selects all elements with a target attribute
<u>[attribute=value]</u>	[target="_blank"]	Selects all elements with target="_blank"
<u>[attribute~=value]</u>	[title~="flower"]	Selects all elements with a title attribute that contains a space-separated list of words, one of which is "flower"
<u>[attribute =value]</u>	[lang "en"]	Selects all elements with a lang attribute value starting with "en"
<u>[attribute^=value]</u>	a[href^="https"]	Selects all <a> elements with a href attribute value starting with "https"
<u>[attribute\$=value]</u>	a[href\$=".pdf"]	Selects all <a> elements with a href attribute value ending with ".pdf"
<u>[attribute*=value]</u>	a[href*="w3schools"]	Selects all <a> elements with a href attribute value containing the substring "w3schools"

https://www.w3schools.com/css/css_attribute_selectors.asp



Ereditarietà



Propagazione degli stili

- Alcune proprietà sono ereditate dai **discendenti**
 - la dichiarazione color viene passa allo span
 - le altre no

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
p{  
  color:white;  
  background:grey;  
  border: medium solid black;  
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
  <h1>This is a Heading</h1>
```

```
  <p>This is a paragraph <span>with a span</span> inside.</p>
```

```
</body>
```

```
</html>
```

This is a Heading

This is a paragraph with a span inside.

Proprietà ereditate



- azimuth
- border-collapse
- border-spacing
- caption-side
- color
- cursor
- direction
- elevation
- empty-cells
- font-family
- font-size
- font-style
- font-variant
- font-weight
- font
- letter-spacing
- line-height
- list-style-image
- list-style-position
- list-style-type
- list-style
- orphans
- pitch-range
- pitch
- quotes
- stress
- text-align
- text-indent
- text-transform
- visibility
- voice-family
- white-space
- widows
- word-spacing



Conflitti



Conflitti

- Più regole possono applicarsi allo stesso elemento
- Il browser deve scegliere quale valore usare
- Questo è un conflitto tra dichiarazioni

```
<p id="intro" class="note">Testo</p>
```

```
p { color: green; }  
.note { color: blue; }  
#intro { color: red; }
```

Di che colore sarà?



Come il browser decide chi vince

- Il browser confronta le dichiarazioni in conflitto
- Alcune regole sono più **forti** di altre
- A parità di forza, vince quella scritta dopo

```
p { color: green; }  
p { color: blue; }
```

```
<p>Testo</p>
```

Il testo sarà blu!!

Esempio 1



```
<!DOCTYPE html>
<html>
<head>
<style>

p{
  color: blue;
  margin-left: 30px;
}

p{
  color: green;
}

</style>
</head>
<body>

<h1>This is a Heading</h1>
<p>This is a paragraph.</p>
<p>This is another paragraph.</p>

</body>
</html>
```

- I paragrafi sono verdi
- Il margine rimane

This is a Heading

This is a paragraph.

This is another paragraph.

```
element.style {
}

p {
  color: ■ green;
}

p {
  color: ■ blue;
  margin-left: 30px;
}
```

devtool

Esempio 2



```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
p{
  color: blue;
  margin-left: 30px;
}
```

```
p{
  color: green;
}
```

```
</style>
</head>
<body>
```

```
<h1>This is a Heading</h1>
```

```
<p>This is a paragraph.</p>
```

```
<p style="color: red;">This is another par
```

```
</body>
```

```
</html>
```

- Il secondo paragrafo è rosso!!
 - domina lo style inline

This is a Heading

This is a paragraph.

This is another paragraph.

```
element.style {
  color: red;
}

p {
  color: green;
}

p {
  ✓ color: blue;
  ✓ margin-left: 30px;
}
```

devtool

Esempio 3



```
<!DOCTYPE html>
<html>
<head>
<style>

#para3{
  color: orange;
}

.red{
  color: red;
}

p{
  color: green;
}

</style>
</head>
<body>

<h1>This is a Heading</h1>
<p>This is a paragraph.</p>
<p class="red">This is a paragraph with class.</p>
<p id="para3" class="red">This is a paragraph with id and class.</p>

</body>
</html>
```

This is a Heading

This is a paragraph.

This is a paragraph with class.

This is a paragraph with id and class.

Non tutti i selettori sono uguali, alcuni hanno più "peso" di altri

Esempio 1



```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
p{
```

```
  color: red;
```

```
}
```

```
p span{
```

```
  color: green;
```

```
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<h1>This is a Heading</h1>
```

```
<p>This is a paragraph.</p>
```

```
<p>This is a <span>paragraph with a span</span> inside.</p>
```

```
</body>
```

```
</html>
```

This is a Heading

This is a paragraph.

This is a paragraph with a span inside.

Esempio 2



```
<!DOCTYPE html>
<html>
<head>
<style>
.news p{
  color: green;
}

.blog p{
  color: red;
}

</style>
</head>
<body>
<h1>This is a Heading</h1>
<section class="news">
  <p>This is a paragraph in sec news.</p>
  <p>This is a another paragraph in sec news.</p>
</section>
<section class="blog">
  <p>This is a paragraph in sec blog.</p>
  <p>This is a another paragraph in sec blog.</p>
</section>
</body>
</html>
```

This is a Heading

This is a paragraph in sec news.

This is a another paragraph in sec news.

This is a paragraph in sec blog.

This is a another paragraph in sec blog.

Esempio 3



```
<!DOCTYPE html>
<html>
<head>
<style>
#news p{
  color: green;
}

#blog p{
  color: red;
}

</style>
</head>
<body>

<h1>This is a Heading</h1>
<section id="news">
  <p>This is a paragraph in sec news.</p>
  <p>This is a another paragraph in sec news.</p>
</section>
<section id="blog">
  <p>This is a paragraph in sec blog.</p>
  <p>This is a another paragraph in sec blog.</p>
</section>
</body>
</html>
```

This is a Heading

This is a paragraph in sec news.

This is a another paragraph in sec news.

This is a paragraph in sec blog.

This is a another paragraph in sec blog.

Esempio 4



```
<!DOCTYPE html>
<html>
<head>
<style>
section{
  color: grey;
}

.news .mark{
  font-weight: bold;
}

.blog .mark{
  color: orange;
}

</style>
</head>
<body>

<h1>This is a Heading</h1>
<section class="news">
  <p>This is a paragraph in sec news.</p>
  <p class="mark">This is a another paragraph in sec news.</p>
</section>
<section class="blog">
  <p class="mark">This is a paragraph in sec blog.</p>
  <p>This is a another <span class="mark">paragraph</span> in sec blog.
</p>
</section>
</body>
</html>
```

This is a Heading

This is a paragraph in sec news.

This is a another paragraph in sec news.

This is a paragraph in sec blog.

This is a another **paragraph** in sec blog.

Esempio 5



```
<!DOCTYPE html>
<html>
<head>
<style>
.news.mark{
  font-weight: bold;
}

.news .mark{
  color: orange;
  font-weight: normal;
}

</style>
</head>
<body>

<h1>This is a Heading</h1>
<section class="news mark">
  <p>This is a paragraph in sec news.</p>
  <p class="mark">This is a another paragraph in sec news.</p>
</section>
</section>
</body>
</html>
```

This is a Heading

This is a paragraph in sec news.

This is a another paragraph in sec news.



Classi multiple

```
<!DOCTYPE html>
<html>
<head>
<style>
.btn {
  font-size: 18px; color: white; margin: 10px;
  padding: 10px; display: block; width: 100px;
}

.btn-danger {
  background: red;
}

.btn-success {
  background: green;
}

</style>
</head>
<body>

<h1>This is a Heading</h1>
<a class="btn btn-danger" href="">Pericolo</a>
<a class="btn btn-success" href="">Hai vinto</a>

</body>
</html>
```

This is a Heading

Pericolo

Hai vinto



Cascade



- Il Cascade "**cascata**" è un algoritmo che definisce come combinare valori di proprietà provenienti da fonti diverse.
- Stili:
 - **del browser** - user-agent stylesheets
 - **dell'autore** - quello che scriviamo noi
 - **dell'utente** - the user of the browser, may have a custom style sheet to tailor its experience.
- La cascata crea delle collisioni che vanno risolte

<https://developer.mozilla.org/it/docs/Web/CSS/Cascade>



Esempio

- il browser applica già stile predefinito a h1, p, a

```
<h1>Titolo</h1>  
<p>Paragrafo</p>  
<a href="#">Link</a>
```

```
a {  
  color: darkred;  
  text-decoration: none;  
}
```

Cascading Order



Precedence Order (low to high)	Origin	Importance
1	user-agent (browser)	normal
2	user	normal
3	author (developer)	normal
4	CSS @keyframe animations	
5	author (developer)	!important
6	user	!important
7	user-agent (browser)	!important
8	CSS transitions	

https://developer.mozilla.org/it/docs/Web/CSS/Cascade#cascading_order



!important

```
<!DOCTYPE html>
<html>
<head>
<style>

#para3{
  color: orange;
}

.red{
  color: red;
}

p{
  color: green !important;
}

</style>
</head>
<body>

<h1>This is a Heading</h1>
<p>This is a paragraph.</p>
<p class="red">This is a paragraph with class.</p>
<p id="para3" class="red">This is a paragraph with
id and class.</p>

</body>
</html>
```

- Se non voglio che una regola venga sovrascritta la posso dichiarare **!important**
- Viene usato in casi molto particolari
 - di solito è sempre possibile evitarlo

This is a Heading

This is a paragraph.

This is a paragraph with class.

This is a paragraph with id and class.



Specificità



Non tutti i selettori hanno lo stesso peso

- Quando più regole competono, il browser confronta il selettore
- Un selettore più specifico prevale su uno più generico

```
p  
  .note  
  #intro
```

- id > classe > elemento
- Più il selettore individua precisamente l'elemento, più “pesa”



Come si calcola la specificità

- La specificità si confronta come una tupla di valori [a,b,c,d]
- Si legge da sinistra a destra
 - Conta prima il valore più importante
- Calcolo dei valori [a,b,c,d]
 - a. 1 se la dichiarazione è inline, 0 altrimenti
 - b. numero di selettori id
 - c. numero di selettori di classe, attributo o pseudo-classe
 - d. numero di selettori elemento o pseudo elemento

```
#intro  
.note  
p
```

```
#intro    -> [0,1,0,0]  
.note     -> [0,0,1,0]  
p         -> [0,0,0,1]
```



Esempi di specificità

```
<p id="intro" class="note important">Benvenuti</p>
```

p	/* 0,0,0,1 */
.note	/* 0,0,1,0 */
p.note	/* 0,0,1,1 */
.note.important	/* 0,0,2,0 */
#intro	/* 0,1,0,0 */
style=""	/* 1,0,0,0 */

Si confronta da sinistra a destra!!

Esempi



- Dallo standard **(6.4.3 Calculating a selector's specificity)**

<code>* {}</code>	<code>/* a=0 b=0 c=0 d=0 -> specificity = 0,0,0,0 */</code>
<code>li {}</code>	<code>/* a=0 b=0 c=0 d=1 -> specificity = 0,0,0,1 */</code>
<code>li::first-line {}</code>	<code>/* a=0 b=0 c=0 d=2 -> specificity = 0,0,0,2 */</code>
<code>ul li {}</code>	<code>/* a=0 b=0 c=0 d=2 -> specificity = 0,0,0,2 */</code>
<code>ul ol+li {}</code>	<code>/* a=0 b=0 c=0 d=3 -> specificity = 0,0,0,3 */</code>
<code>h1 + *[rel=up]{}</code>	<code>/* a=0 b=0 c=1 d=1 -> specificity = 0,0,1,1 */</code>
<code>ul ol li.red {}</code>	<code>/* a=0 b=0 c=1 d=3 -> specificity = 0,0,1,3 */</code>
<code>li.red.level {}</code>	<code>/* a=0 b=0 c=2 d=1 -> specificity = 0,0,2,1 */</code>
<code>#x34y {}</code>	<code>/* a=0 b=1 c=0 d=0 -> specificity = 0,1,0,0 */</code>
<code>style=""</code>	<code>/* a=1 b=0 c=0 d=0 -> specificity = 1,0,0,0 */</code>



Esempi

- Dallo standard **(6.4.3 Calculating a selector's specificity)**

<code>* {}</code>	<code>/* a=0 b=0 c=0 d=0 -> specificity = 0,0,0,0 */</code>
<code>li html body div#pagewrap ul#summer-drinks li.favorite</code>	
<code>li::first-line {}</code>	<code>/* a=0 b=0 c=0 d=2 -> specificity = 0,0,0,2 */</code>
<code>ul li {}</code>	<code>/* a=0 b=0 c=0 d=2 -> specificity = 0,0,0,2 */</code>
<code>ul ol+li {}</code>	<code>/* a=0 b=0 c=0 d=3 -> specificity = 0,0,0,3 */</code>
<code>h1 + *[rel=up]{} </code>	<code>/* a=0 b=0 c=1 d=1 -> specificity = 0,0,1,1 */</code>
<code>ul ol li.red {}</code>	<code>/* a=0 b=0 c=1 d=3 -> specificity = 0,0,1,3 */</code>
<code>li.red.level {}</code>	<code>/* a=0 b=0 c=2 d=1 -> specificity = 0,0,2,1 */</code>
<code>#x34y {}</code>	<code>/* a=0 b=1 c=0 d=0 -> specificity = 0,1,0,0 */</code>
<code>style=""</code>	<code>/* a=1 b=0 c=0 d=0 -> specificity = 1,0,0,0 */</code>